

MASTER INTELLIGENCE ARTIFICIELLE

Devoir pratique — Spring Boot & Angular (PrimeNG 17)

RAPPORT DE PROJET

Plateforme de gestion de datasets et de modèles de Machine Learning

Binôme : KEITA_FAYCAL ET SAWADOGO_FAYCAL

Année universitaire 2026/2027

1. Contexte et objectifs

Ce projet répond au devoir pratique du Master Intelligence Artificielle portant sur le développement full-stack avec Spring Boot côté serveur et Angular associé à PrimeNG 17 côté client. Un laboratoire de recherche fictif souhaite disposer d'un outil interne permettant à ses chercheurs de cataloguer les jeux de données (datasets) qu'ils utilisent, les modèles de Machine Learning qu'ils entraînent, et les expérimentations qui associent un modèle à un dataset avec ses métriques de performance.

L'application développée couvre l'ensemble de la chaîne demandée : une API REST structurée en couches, une persistance relationnelle via Spring Data JPA, une validation stricte des données, une gestion centralisée des erreurs, une documentation interactive via Swagger, et une interface Angular complète consommant cette API à l'aide des composants PrimeNG (tableaux, formulaires réactifs, boîtes de dialogue).

2. Modèle de données

Le modèle de données comprend trois entités liées : Dataset, ModeleML et Experimentation. Une Experimentation référence exactement un Dataset et un ModeleML (relations @ManyToOne côté Experimentation) ; un Dataset comme un ModeleML peuvent être utilisés par plusieurs expérimentations (relations @OneToMany inverses, mappedBy). Cette structure permet de tracer précisément quelles combinaisons dataset/modèle ont été testées et avec quels résultats.

Entité	Attributs principaux	Relations	Table SQL
Dataset	id, nom, description, source, nombreobservations, format (enum : CSV/JSON/images), dateAjout	1 Dataset → N Experimentation (@OneToMany, mappedBy = "dataset")	dataset
ModeleML	id, nom, typeml (enum : classification/régression/clustering), algorithme, version, dateCreation	1 ModeleML → N Experimentation (@OneToMany, mappedBy = "modele")	modeleml
Experimentation	id, accuracy [0-1], f1Score [0-1], dureeEntrainement, dateExecution	N Experimentation → 1 Dataset N Experimentation → 1 ModeleML (@ManyToOne + @JoinColumn)	experimentation

Tableau 1 — Entités, attributs principaux et relations du modèle de données.

Les bornes métier sont imposées dès la couche de validation : accuracy et f1Score doivent être compris entre 0 et 1 (@DecimalMin/@DecimalMax), et les champs numériques comme nombreobservations ou dureeEntrainement ne peuvent être négatifs (@PositiveOrZero). Les énumérations (format du dataset, type de modèle) sont stockées en base sous forme de texte (EnumType.STRING) plutôt que d'index numérique, ce qui garde le contenu de la base lisible et évite une corruption des données existantes si l'ordre des valeurs de l'enum venait à changer.

3. Architecture backend

3.1 Séparation en couches

L'architecture respecte une séparation stricte en quatre couches, conformément aux exigences du sujet :

- Contrôleur — expose les endpoints REST et délègue systématiquement au service correspondant, sans aucune logique métier.
- Service (interface + implémentation) — contient la logique métier : recherche, validation d'existence, mapping entité ↔ DTO.
- Repository — interfaces Spring Data JPA (JpaRepository), CRUD généré automatiquement sans SQL à écrire.
- Entity / DTO — les entités JPA modélisent la persistance ; des DTO séparés (DatasetDTO, ModeleMLDTO, ExperimentationDTO) définissent le contrat exposé par l'API.

Le choix d'introduire des DTO plutôt que d'exposer directement les entités JPA répond à deux problèmes concrets : éviter la sérialisation en boucle infinie provoquée par les relations bidirectionnelles (Dataset → Experimentation → Dataset → ...), et découpler le contrat de l'API du modèle de persistance, qui peut évoluer indépendamment.

3.2 API REST

Chaque entité expose un jeu complet de cinq endpoints, avec les codes de statut HTTP appropriés :

Méthode	URL	Description
GET	/api/{ressource}	Liste de toutes les ressources
GET	/api/{ressource}/{id}	Détail d'une ressource
POST	/api/{ressource}	Création (201 Created)
PUT	/api/{ressource}/{id}	Modification
DELETE	/api/{ressource}/{id}	Suppression (204 No Content)

Tableau 2 — Structure commune des endpoints REST, déclinée pour /api/datasets, /api/modeles et /api/experimentations.

3.3 Validation et gestion des erreurs

La validation des données d'entrée s'appuie sur jakarta.validation (@NotBlank, @NotNull, @DecimalMin, @DecimalMax, @PositiveOrZero) appliquée directement sur les DTO et déclenchée via l'annotation @Valid sur les paramètres @RequestBody des contrôleurs.

La gestion des erreurs est centralisée dans une classe unique annotée @RestControllerAdvice, qui intercepte trois catégories d'exceptions : ResourceNotFoundException (404, ressource introuvable), MethodArgumentNotValidException (400, échec de validation, avec le détail des erreurs par champ), et Exception générique (500, filet de sécurité). Toutes les réponses d'erreur suivent un format structuré identique : code, message et horodatage, comme l'exige le sujet.

3.4 Documentation et configuration transverse

La documentation de l'API est générée automatiquement par springdoc-openapi-starter-webmvc-ui et accessible via une interface Swagger interactive (/swagger-ui.html), avec une spécification OpenAPI exposée en JSON (/v3/api-docs).

Le frontend Angular et le backend Spring Boot fonctionnant sur deux origines différentes (respectivement les ports 4200 et 8080), une configuration CORS explicite (WebMvcConfigurer) autorise les requêtes du frontend vers /api/** — sans cette configuration, le navigateur bloquerait par défaut tous les appels HttpClient d'Angular vers l'API.

4. Architecture frontend

4.1 Structure Angular

Le frontend est un projet Angular 17 utilisant des composants standalone (sans NgModule racine) et un routing avec chargement différé (loadComponent), afin que chaque vue ne soit téléchargée par le navigateur qu'au moment de la navigation vers elle. La navigation s'organise autour de trois vues correspondant aux trois entités : Datasets, Modèles ML et Expérimentations.

- models/ — interfaces TypeScript miroir des DTO exposés par le backend.
- services/ — un service HttpClient dédié par entité, encapsulant les appels GET/POST/PUT/DELETE.
- pages/ — un composant par vue, combinant tableau, formulaire réactif et boîtes de dialogue.

4.2 Composants PrimeNG utilisés

Chaque vue s'appuie sur le même ensemble de composants PrimeNG 17, garantissant une expérience homogène :

- p-table — pagination, tri par colonne et recherche globale sur chaque tableau.
- p-dialog — formulaire réactif de création et de modification, avec retour visuel des erreurs de validation (champs requis, plages numériques).
- p-confirmDialog — confirmation systématique avant toute suppression.
- p-toast — notifications de succès ou d'échec après chaque action.
- p-dropdown, p-calendar, p-inputNumber — saisie contrainte pour les énumérations, dates et valeurs numériques.

La vue Expérimentations présente une particularité : elle charge au démarrage la liste complète des datasets et des modèles ML pour alimenter deux listes déroulantes, permettant à l'utilisateur de sélectionner un dataset et un modèle par leur nom tout en transmettant leur identifiant au backend.

5. Difficultés rencontrées et solutions apportées

Plusieurs difficultés concrètes ont été rencontrées et résolues au cours du développement et des tests. Elles sont détaillées ci-dessous car elles illustrent des points de vigilance transférables à d'autres projets Spring Boot / Angular.

5.1 Entité Experimentation incomplète

Problème — L'entité Experimentation issue de la mise en place initiale du projet ne compilait pas : les champs métier (accuracy, f1Score, dureeEntrainement, dateExecution) avaient été remplacés par du texte collé sans déclaration Java valide, et aucune relation inverse (@OneToMany) n'existait côté Dataset et ModeleML.

Solution — Les champs ont été redéclarés avec leurs types et leurs contraintes de validation (@DecimalMin("0.0")/@DecimalMax("1.0")) pour les métriques comprises entre 0 et 1, @PositiveOrZero pour la durée, et les relations @OneToMany(mappedBy = ...) ont été ajoutées sur Dataset et ModeleML pour compléter le modèle bidirectionnel exigé par le sujet.

5.2 Échec de création via l'API (erreur 500)

Problème — Lors des tests de bout en bout, toute requête POST de création échouait avec une erreur 500 : "Cannot map null into type int". La cause identifiée est l'utilisation d'un champ id de type int primitif dans les DTO, combinée à Lombok (@AllArgsConstructor) et Jackson : lorsque le client ne transmet pas d'id à la création (cas normal), Jackson tente de désérialiser null vers un type primitif, ce qui provoque l'exception.

Solution — Le champ id des trois DTO (DatasetDTO, ModeleMLDTO, ExperimentationDTO) a été migré de int vers Integer, qui accepte null. Les méthodes de mapping DTO → entité ont également été revues pour ne plus recopier un id potentiellement nul avant sa réinitialisation explicite à la création.

5.3 Blocage CORS entre le frontend et le backend

Problème — Le frontend Angular (port 4200) et le backend Spring Boot (port 8080) constituant deux origines distinctes, les appels HttpClient étaient bloqués par la politique same-origin du navigateur.

Solution — Une classe de configuration WebMvcConfigurer dédiée autorise explicitement les requêtes GET/POST/PUT/DELETE/OPTIONS depuis http://localhost:4200 vers les routes /api/**, sans ouvrir l'API à d'autres origines.

5.4 Interaction des composants PrimeNG (calendrier, liste déroulante) avec les formulaires réactifs

Problème — Les composants complexes de PrimeNG (p-calendar, p-dropdown) encapsulent leur propre logique interne de saisie et de formatage, qu'il fallait valider en conditions réelles pour s'assurer qu'ils se synchronisent correctement avec les FormControl du formulaire réactif Angular.

Solution — Les trois formulaires ont été testés de bout en bout dans le navigateur (sélection de date, sélection dans les listes déroulantes, saisie numérique), confirmant que le binding via ReactiveFormsModule et l'implémentation ControlValueAccessor de PrimeNG fonctionnent correctement une fois les événements natifs du navigateur déclenchés.

6. Tests et validation

L'application a été testée de bout en bout avant le rendu, à deux niveaux complémentaires.

6.1 Tests de l'API (backend)

- CRUD complet vérifié sur les trois entités : création (201), lecture (200), modification (200), suppression (204).
- Validation des bornes métier vérifiée : une requête avec accuracy = 1.5 renvoie bien une erreur 400 structurée, avec le détail du champ en cause.
- Gestion des ressources manquantes vérifiée : une requête sur un id inexistant renvoie une erreur 404 avec un message explicite.
- Documentation Swagger accessible et fonctionnelle sur /swagger-ui.html.

6.2 Tests de l'interface (frontend)

- Création d'un dataset via le formulaire complet (champs texte, liste déroulante de format, sélecteur de date) suivie de son apparition immédiate dans le tableau.
- Suppression avec confirmation testée : la boîte de dialogue de confirmation s'affiche, et la suppression n'est effective qu'après validation.
- Les trois vues (Datasets, Modèles ML, Expérimentations) affichent correctement les données réelles issues de l'API, y compris les noms de dataset et de modèle liés dans le tableau des expérimentations.

Un parcours de démonstration recommandé consiste à présenter Swagger, puis à créer un dataset, un modèle et une expérimentation les reliant depuis l'interface Angular, avant de montrer une modification, une suppression confirmée, et un cas d'erreur de validation.

7. Installation et lancement

Chaque dossier du projet (backend et frontend) contient son propre fichier README.md, autonome, détaillant prérequis, étapes d'installation et particularités. Les instructions essentielles sont résumées ci-dessous.

7.1 Backend — datasetsMachineLearning/

Prérequis : JDK 21, PostgreSQL 14+ installé et démarré localement (le wrapper Maven fourni évite d'installer Maven séparément).

```
1. Créer la base : psql -U postgres -h localhost -c "CREATE DATABASE datasets_ml;"
```

```
2. Adapter spring.datasource.username / password dans
src/main/resources/application.properties
3. Lancer : ./mvnw spring-boot:run
4. API disponible sur http://localhost:8080
5. Documentation interactive : http://localhost:8080/swagger-ui.html
```

Le schéma des tables est créé automatiquement au démarrage (spring.jpa.hibernate.ddl-auto=update) : aucun script SQL à exécuter manuellement au-delà de la création de la base elle-même.

7.2 Frontend — Angulardataset/

Prérequis : Node.js 18+ et npm. Le backend doit être démarré au préalable, le frontend ne contenant aucune donnée en dur.

```
1. Installer les dépendances : npm install
2. Lancer le serveur de développement : npm start
3. Application disponible sur http://localhost:4200
4. Build de production : npm run build (sortie dans dist/angulardataset/)
```

Le backend autorise par défaut les requêtes CORS depuis http://localhost:4200 ; les deux applications doivent donc tourner simultanément, chacune sur son port par défaut, pour que l'interface puisse consommer l'API.

8. Conclusion

Ce projet a permis de mettre en œuvre l'ensemble des compétences visées par le devoir : conception d'une API REST structurée en couches avec Spring Boot et Spring Data JPA, modélisation d'entités liées par des relations JPA bidirectionnelles, validation des données et gestion centralisée des erreurs, documentation interactive via Swagger, et construction d'une interface Angular complète consommant cette API à l'aide des composants PrimeNG.

Les difficultés rencontrées — une entité incomplète, un bug de désérialisation JSON, une configuration CORS manquante — ont toutes été diagnostiquées et corrigées grâce à des tests systématiques de bout en bout, aussi bien côté API que côté interface. L'application est aujourd'hui fonctionnelle et prête pour la démonstration.